



Tree Problem Patterns — Complete Beginner Notes

Goal: Learn the pattern, not the problem. Solve any tree question by recognising which pattern it belongs to.



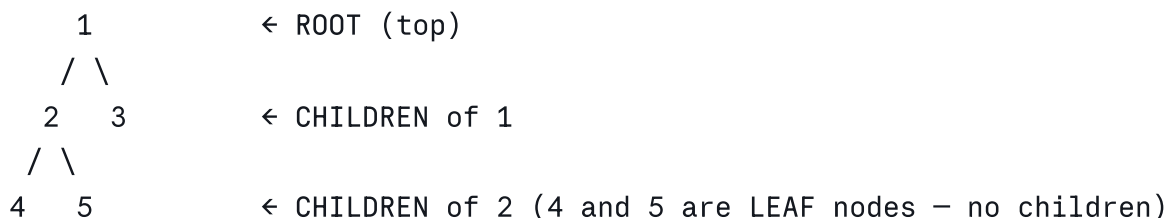
Table of Contents

1. [Tree Basics — Build the Mental Model First](#)
 2. [The TreeNode Class \(Always the Same\)](#)
 3. [The 5 Core Traversals](#)
 4. [Pattern 1 — Simple DFS \(Bottom-Up Return Value\)](#)
 5. [Pattern 2 — Comparing Two Trees](#)
 6. [Pattern 3 — BFS / Level Order Traversal](#)
 7. [Pattern 4 — BST Properties](#)
 8. [Pattern 5 — Path Problems \(Global Max\)](#)
 9. [Pattern 6 — Tree Construction](#)
 10. [Pattern 7 — Serialization & Deserialization](#)
 11. [How to Identify the Pattern \(Decision Tree\)](#)
 12. [Complexity Cheat Sheet](#)
 13. [Common Mistakes to Avoid](#)
-

1. Tree Basics

What is a Tree?

A tree is a collection of **nodes** connected by **edges**, with one special node called the **root**.



Vocabulary You Must Know

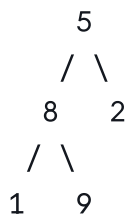
Term	Meaning	Example above
Root	Top-most node	Node 1

Term	Meaning	Example above
Leaf	Node with NO children	Nodes 3, 4, 5
Parent	Node directly above	1 is parent of 2 and 3
Child	Node directly below	2 and 3 are children of 1
Height	Longest path from node to a leaf	Height of tree = 3
Depth	Distance from root to a node	Depth of node 4 = 3
Subtree	A node + all its descendants	Node 2 with nodes 4 & 5

Binary Tree vs Binary Search Tree (BST)

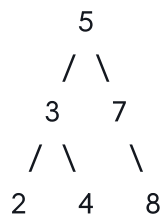
Binary Tree

Any value anywhere



Binary Search Tree (BST)

LEFT < ROOT < RIGHT ← ALWAYS!



BST Rule: For every node, ALL values in the left subtree are SMALLER, and ALL values in the right subtree are LARGER.

2. The TreeNode Class

This class is ALWAYS given to you in LeetCode. Just memorise it.

```
java
```

```
// You do NOT write this – it's given. But understand it.
public class TreeNode {
    int val;
    TreeNode left;    // pointer to left child
    TreeNode right;   // pointer to right child

    // Constructors
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }
}
```

How a Tree is Stored in Memory

```
TreeNode root = new TreeNode(1);
root.left  = new TreeNode(2);
root.right = new TreeNode(3);
```

```

root
 |
[1]
 / \
[2] [3]
```

`root.left` gives you node 2. `root.left.val` gives you 2. `root.left.left` is **null** (no child).

3. The 5 Core Traversals

These are the building blocks. Every tree problem uses one of these.

3a. DFS — Depth First Search (Go DEEP before going wide)

There are 3 DFS orders. The only difference is **when you process the current node**.



Order	Pattern	Visit Sequence	Use For
Pre-order	Root → Left → Right	1 → 2 → 4 → 5 → 3	Copying/serialising a tree
In-order	Left → Root → Right	4 → 2 → 5 → 1 → 3	BST → gives SORTED order
Post-order	Left → Right → Root	4 → 5 → 2 → 3 → 1	Deleting/computing bottom-up

🔑 MASTER RECURSIVE DFS TEMPLATE (Use this for almost everything!)

```

java

// THE UNIVERSAL RECURSIVE DFS TEMPLATE
void dfs(TreeNode node) {
    // BASE CASE – always check null first!
    if (node == null) return;

    // PRE-ORDER: process node HERE → before going left/right
    // process(node.val);

    dfs(node.left);    // go left

    // IN-ORDER: process node HERE → between left and right
    // process(node.val);

    dfs(node.right);   // go right

    // POST-ORDER: process node HERE → after left and right
    // process(node.val);
}
  
```

How the call stack works visually:

```

dfs(1)
  → dfs(2)
    → dfs(4)
      → dfs(null) ← base case, return
      → dfs(null) ← base case, return
    ← back to dfs(4), done
  → dfs(5)
    → dfs(null) ← base case, return
    → dfs(null) ← base case, return
  ← back to dfs(5), done
← back to dfs(2), done
→ dfs(3)
  → dfs(null) ← base case, return
  → dfs(null) ← base case, return
← back to dfs(3), done
← back to dfs(1), done

```

3b. BFS — Breadth First Search (Go WIDE, level by level)

```

      1           ← Level 0
     / \
    2   3         ← Level 1
   / \
  4   5           ← Level 2

```

BFS visits: 1 → 2 → 3 → 4 → 5

MASTER BFS TEMPLATE

```
java
```

```
// THE UNIVERSAL BFS / LEVEL ORDER TEMPLATE
void bfs(TreeNode root) {
    if (root == null) return;

    Queue queue = new LinkedList<>();
    queue.offer(root); // start with root in queue

    while (!queue.isEmpty()) {
        int levelSize = queue.size(); // how many nodes at this level

        for (int i = 0; i < levelSize; i++) {
            TreeNode node = queue.poll(); // take node out

            // ✅ PROCESS node here
            System.out.println(node.val);

            // Add children to queue for next level
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        // ← one full level is done here
    }
}
```

Key insight: The `for (int i = 0; i < levelSize; i++)` loop isolates one level at a time. Remove it if you don't need level-by-level processing.

PATTERN 1 — Simple DFS (Bottom-Up Return Value)

When to Use

- Questions asking for a **single value** computed from the whole tree
- Examples: max depth, diameter, is balanced, count nodes
- Key signal: the answer at a node **depends on answers from its children**

The Mental Model

Think BOTTOM-UP. Leaves answer first, root answers last.

```

    ?           ← "What is my answer? I'll ask my children first."
  / \
 ?   ?       ← "What is MY answer? I'll ask MY children first."
```

```
 / \
?   ?      ← "I'm a leaf. My answer is the base case."
```

🔑 PATTERN 1 TEMPLATE

java

```
// PATTERN 1: DFS with return value (bottom-up)
int solve(TreeNode node) {
    // STEP 1: Base case (null node)
    if (node == null) return /* base value, usually 0 or some identity */;

    // STEP 2: Recurse – get answers from children FIRST
    int leftResult = solve(node.left);
    int rightResult = solve(node.right);

    // STEP 3: Combine children's answers to get THIS node's answer
    int myAnswer = /* some combination of leftResult, rightResult, node.val */;

    // STEP 4: Return YOUR answer upward
    return myAnswer;
}
```

Example A — Maximum Depth of Binary Tree

Problem: Find the longest path from root to any leaf.

```
      3          depth = 3
     / \
    9  20
     / \
    15  7
```

Answer: 3

Thinking process:

Q: "What is the max depth of tree rooted at 3?"

A: $1 + \max(\text{depth of left subtree}, \text{depth of right subtree})$
 $= 1 + \max(\text{depth of 9}, \text{depth of 20})$

Q: "What is depth of 9?" $\rightarrow$ it's a leaf $\rightarrow$ depth = 1

Q: "What is depth of 20?" $\rightarrow 1 + \max(\text{depth of 15}, \text{depth of 7}) = 1 + 1 = 2$

Answer: $1 + \max(1, 2) = 3$ ✓

java

```
class Solution {
    public int maxDepth(TreeNode root) {
        // Base case: empty tree has depth 0
        if (root == null) return 0;

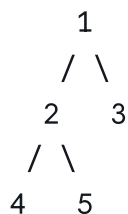
        // Get depth of left and right subtrees
        int leftDepth = maxDepth(root.left);
        int rightDepth = maxDepth(root.right);

        // My depth = 1 (for myself) + deeper of my two children
        return 1 + Math.max(leftDepth, rightDepth);
    }
}

// Time: O(n) - visit every node once
// Space: O(h) - call stack = height of tree
```

Example B — Diameter of Binary Tree

Problem: Longest path between ANY two nodes (path doesn't need to go through root).



Diameter = 3 (path: 4 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3 OR 5 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3)

Key Insight:

At EVERY node, the diameter PASSING THROUGH that node
= leftHeight + rightHeight

The GLOBAL diameter = max of all diameters at each node.

java

```
class Solution {
    int globalMax = 0; // ← stores the answer (global variable)

    public int diameterOfBinaryTree(TreeNode root) {
        height(root); // kick off DFS
        return globalMax;
    }

    // Returns HEIGHT of tree rooted at node
    // As a SIDE EFFECT, updates globalMax
    private int height(TreeNode node) {
        if (node == null) return 0;

        int leftH = height(node.left);
        int rightH = height(node.right);

        // Diameter AT this node = left height + right height
        globalMax = Math.max(globalMax, leftH + rightH);

        // Return height to parent
        return 1 + Math.max(leftH, rightH);
    }
}

// Time: O(n) | Space: O(h)
```

⚠ **Pattern trick:** When you need a "global max" that updates at every node, use an **instance variable** (like `globalMax`) alongside your recursive helper. The helper returns one thing (e.g., height) but ALSO updates the global answer as a side effect.

Example C — Balanced Binary Tree

Problem: Is the tree height-balanced? (No subtree differs in height by more than 1)

java

```

class Solution {
    public boolean isBalanced(TreeNode root) {
        return checkHeight(root) != -1; // -1 = "not balanced" signal
    }

    // Returns height if balanced, -1 if NOT balanced
    private int checkHeight(TreeNode node) {
        if (node == null) return 0; // empty tree is balanced, height = 0

        int leftH = checkHeight(node.left);
        int rightH = checkHeight(node.right);

        // If either child is unbalanced, propagate -1 upward
        if (leftH == -1 || rightH == -1) return -1;

        // Check balance at this node
        if (Math.abs(leftH - rightH) > 1) return -1; // unbalanced!

        // Balanced – return actual height
        return 1 + Math.max(leftH, rightH);
    }
}

// Time: O(n) | Space: O(h)

```

⚠ **Pattern trick:** Use a **sentinel value** (-1) to propagate "failure" up the tree. If any node is unbalanced, -1 bubbles all the way up.

Example D — Invert Binary Tree

Problem: Mirror the tree (swap all left and right children).

BEFORE:		AFTER:
4		4
/ \		/ \
2 7	→	7 2
/ \ / \		/ \ / \
1 3 6 9		9 6 3 1

java

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null; // base case

        // Swap left and right children
        TreeNode temp = root.left;
        root.left = root.right;
        root.right = temp;

        // Recurse into both children
        invertTree(root.left);
        invertTree(root.right);

        return root;
    }
}
// Time: O(n) | Space: O(h)
```

PATTERN 2 — Comparing Two Trees

When to Use

- Two trees (or two subtrees) are given and you need to **compare** them
- Examples: Same Tree, Subtree of Another Tree

The Mental Model

Two trees are "the same" if:

1. Both are null → true
2. One is null, one isn't → false
3. Both non-null: values match AND left subtrees match AND right subtrees match

PATTERN 2 TEMPLATE

java

```

boolean compare(TreeNode p, TreeNode q) {
    // STEP 1: Both null → identical (base case, success)
    if (p == null && q == null) return true;

    // STEP 2: One null, one not → not identical (base case, failure)
    if (p == null || q == null) return false;

    // STEP 3: Values differ → not identical
    if (p.val != q.val) return false;

    // STEP 4: Recurse – check both subtrees
    return compare(p.left, q.left) && compare(p.right, q.right);
}

```

Example A — Same Tree

```

java

class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        if (p.val != q.val) return false;
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}

// Time: O(n) | Space: O(h)

```

Example B — Subtree of Another Tree

Problem: Is `subRoot` a subtree of `root`?

Thinking:

For EVERY node in root, check: "Does my subtree match subRoot?"
 If yes at any node → return true.

```

java

```

```

class Solution {
    public boolean isSubtree(TreeNode root, TreeNode subRoot) {
        // Base case: if root is null, subRoot can't be a subtree
        if (root == null) return false;

        // Check: does tree rooted at THIS node match subRoot?
        if (isSameTree(root, subRoot)) return true;

        // Otherwise, check left and right subtrees
        return isSubtree(root.left, subRoot) || isSubtree(root.right, subRoot);
    }

    // Reuse Pattern 2 template
    private boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        if (p.val != q.val) return false;
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}

// Time: O(n * m) where n = root nodes, m = subRoot nodes
// Space: O(h)

```

PATTERN 3 — BFS / Level Order Traversal

When to Use

- Questions about **levels** of a tree
- "Right side view", "level by level output", "zigzag traversal"
- Any time the word "**level**" appears

🔑 PATTERN 3 TEMPLATE (Revisited with Level Tracking)

```
java
```

```

List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>(); // current level's values

        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);                // collect this level

            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level); // done with this level
    }
    return result;
}
// Time: O(n) | Space: O(n) – queue holds one level at a time

```

Example — Binary Tree Right Side View

Problem: What values do you see looking at the tree from the RIGHT?

```

    1  ←  see 1
   / \
  2   3 ←  see 3
   \
    5  ←  see 5

```

Answer: [1, 3, 5]

Insight: The rightmost node in each level is what you see.

java

```

class Solution {
    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();

                // Last node in this level = rightmost visible node
                if (i == size - 1) result.add(node.val);

                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
        }
        return result;
    }
}
// Time: O(n) | Space: O(n)

```

PATTERN 4 — BST Properties

When to Use

- The tree is a **Binary Search Tree**
- Keywords: "validate BST", "kth smallest", "lowest common ancestor in BST"
- Uses the fundamental BST property: **left < node < right**

BST Mental Models

IN-ORDER traversal of a BST = sorted ascending list!

BST: In-order visits:

```

    4                      2, 3, 4, 6, 7, 8 ← always sorted!
   / \
  2   6

```

```
\ / \
3 5 7
```

Example A — Validate Binary Search Tree

Problem: Is the tree a valid BST?

Wrong approach ❌: Just check $\text{left.val} < \text{root.val} < \text{right.val}$ at each node.

```
    5
   / \
  1  6    ← 6 > 5 ✓ locally looks fine
   / \
  3  7    ← BUT 3 < 5 which is root, INVALID!
```

Right approach ✅: Every node must be within a valid range.

```
java
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    // Every node must satisfy: min < node.val < max
    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true; // null = valid

        // Check this node's value is in range
        if (node.val <= min || node.val >= max) return false;

        // Left subtree: upper bound becomes this node's value
        // Right subtree: lower bound becomes this node's value
        return validate(node.left, min, node.val) &&
            validate(node.right, node.val, max);
    }
}
// Time: O(n) | Space: O(h)
```

Visual of range narrowing:

```

validate(5, -∞, +∞)
├─ validate(1, -∞, 5)  ← left child must be < 5
|   ├─ validate(null, -∞, 1)
|   └─ validate(null, 1, 5)
└─ validate(6, 5, +∞)  ← right child must be > 5
    ├─ validate(3, 5, 6)  ← 3 ≤ 5, INVALID! ×
    └─ validate(7, 6, +∞)

```

Example B — Kth Smallest Element in a BST

Insight: In-order traversal gives sorted order → kth node visited = kth smallest!

```

java

class Solution {
    int count = 0;
    int result = 0;

    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }

    private void inorder(TreeNode node, int k) {
        if (node == null) return;

        inorder(node.left, k); // go left first (smallest values)

        count++;                // visiting this node
        if (count == k) {        // is this the kth?
            result = node.val;
            return;
        }

        inorder(node.right, k); // go right
    }
}

// Time: O(k + h) | Space: O(h)

```

Example C — Lowest Common Ancestor of a BST

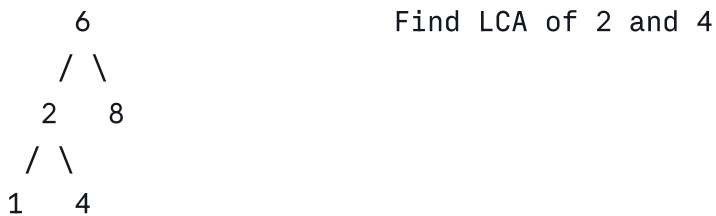
Problem: Find the deepest node that is an ancestor of BOTH p and q.

BST LCA Insight:

If both p and q are LESS than root $\rightarrow$ LCA is in LEFT subtree

If both p and q are GREATER than root $\rightarrow$ LCA is in RIGHT subtree

Otherwise $\rightarrow$ root IS the LCA (they split here!)



Step 1: root = 6. Both 2 and 4 < 6 $\rightarrow$ go left

Step 2: root = 2. 2 $\leq$ 2 and 4 $\geq$ 2 $\rightarrow$ SPLIT! LCA = 2

java

```
class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q)
        // Both in left subtree
        if (p.val < root.val && q.val < root.val)
            return lowestCommonAncestor(root.left, p, q);

        // Both in right subtree
        if (p.val > root.val && q.val > root.val)
            return lowestCommonAncestor(root.right, p, q);

        // Split point  $\rightarrow$  root is the LCA
        return root;
    }
}
// Time: O(h) | Space: O(h)
```

Example D — Count Good Nodes in Binary Tree

Problem: A node is "good" if no node on the path from root to that node has a value GREATER than it.



Good nodes: 3 (root), 3 (left-left), 4, 5 → Answer: 4

Pattern: DFS while tracking the MAX value seen on path from root.

```

java

class Solution {
    public int goodNodes(TreeNode root) {
        return dfs(root, Integer.MIN_VALUE);
    }

    private int dfs(TreeNode node, int maxSoFar) {
        if (node == null) return 0;

        // Is this node "good"?
        int count = (node.val >= maxSoFar) ? 1 : 0;

        // Update max for the path going downward
        int newMax = Math.max(maxSoFar, node.val);

        // Count good nodes in subtrees
        count += dfs(node.left, newMax);
        count += dfs(node.right, newMax);

        return count;
    }
}

// Time: O(n) | Space: O(h)

```

PATTERN 5 — Path Problems (Global Max)

When to Use

- Finding the maximum/minimum along a **path** in the tree
- Path can go through ANY node, not necessarily through root
- Example: Binary Tree Maximum Path Sum

The Mental Model

At each node, a "path through me" looks like:

[some left nodes] → ME → [some right nodes]

left contribution + node.val + right contribution

BUT: I can only pass ONE direction upward to my parent.

So I return: node.val + max(left, right, 0)

🔑 PATTERN 5 TEMPLATE

java

```
int globalMax = Integer.MIN_VALUE; // global answer

int pathGain(TreeNode node) {
    if (node == null) return 0;

    // Max gain from left child (ignore if negative → take 0)
    int leftGain = Math.max(pathGain(node.left), 0);
    int rightGain = Math.max(pathGain(node.right), 0);

    // Best path THROUGH this node
    int pathThroughMe = leftGain + node.val + rightGain;
    globalMax = Math.max(globalMax, pathThroughMe); // update global

    // Return the best SINGLE direction to parent
    return node.val + Math.max(leftGain, rightGain);
}
```

Example — Binary Tree Maximum Path Sum

```

  -10
 /  \
9    20
 /  \
15   7
```

Best path: 15 → 20 → 7 = 42

java

```

class Solution {
    int globalMax = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        maxGain(root);
        return globalMax;
    }

    private int maxGain(TreeNode node) {
        if (node == null) return 0;

        // Only take child contribution if it's positive
        int leftGain = Math.max(maxGain(node.left), 0);
        int rightGain = Math.max(maxGain(node.right), 0);

        // Path through this node
        globalMax = Math.max(globalMax, leftGain + node.val + rightGain);

        // Return best contribution to parent (can only pick one direction)
        return node.val + Math.max(leftGain, rightGain);
    }
}

// Time: O(n) | Space: O(h)

```

PATTERN 6 — Tree Construction

When to Use

- Given **traversal arrays** (preorder, inorder), **reconstruct** the tree

Key Insight About Traversals

Preorder = [ROOT, left subtree, right subtree]
 Inorder = [left subtree, ROOT, right subtree]

Given:

Preorder: [3, 9, 20, 15, 7]

Inorder: [9, 3, 15, 20, 7]

Step 1: preorder[0] = 3 → ROOT is 3

Step 2: Find 3 in inorder → index 1

inorder left of 3: [9] → left subtree has 1 node

inorder right of 3: [15, 20, 7] → right subtree has 3 nodes
Step 3: Recurse!



🔑 PATTERN 6 TEMPLATE

```
java

// Store inorder indices in a map for O(1) lookup
Map<Integer, Integer> inorderIndex = new HashMap<>();
int[] preorder; // global reference

TreeNode build(int preStart, int inStart, int inEnd) {
    if (preStart >= preorder.length || inStart > inEnd) return null;

    // Root = first element of current preorder slice
    int rootVal = preorder[preStart];
    TreeNode root = new TreeNode(rootVal);

    // Find root in inorder – splits left and right subtrees
    int mid = inorderIndex.get(rootVal);
    int leftSize = mid - inStart; // number of nodes in left subtree

    // Build left and right subtrees recursively
    root.left = build(preStart + 1, inStart, mid - 1);
    root.right = build(preStart + leftSize + 1, mid + 1, inEnd);

    return root;
}
```

Example — Construct Binary Tree from Preorder and Inorder Traversal

```
java
```

```

class Solution {
    Map<Integer, Integer> inorderIndex = new HashMap<>();
    int[] preorder;

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        this.preorder = preorder;

        // Pre-build the inorder index map for O(1) lookup
        for (int i = 0; i < inorder.length; i++)
            inorderIndex.put(inorder[i], i);

        return build(0, 0, inorder.length - 1);
    }

    private TreeNode build(int preStart, int inStart, int inEnd) {
        if (preStart >= preorder.length || inStart > inEnd) return null;

        int rootVal = preorder[preStart];
        TreeNode root = new TreeNode(rootVal);

        int mid = inorderIndex.get(rootVal); // position in inorder
        int leftSize = mid - inStart;

        root.left = build(preStart + 1, inStart, mid - 1);
        root.right = build(preStart + leftSize + 1, mid + 1, inEnd);

        return root;
    }
}
// Time: O(n) | Space: O(n)

```

PATTERN 7 — Serialization & Deserialization

When to Use

- Convert a tree to a string (serialize) and back (deserialize)
- Hardest pattern — combines DFS + string parsing

The Approach

Serialise (tree → string):
 Pre-order DFS. Write node value. Write "#" for null.



Serialised: "1,2,##,3,4,##,5,##"

Deserialise (string → tree):

Read values one by one. "#" = null. Build tree in same pre-order.

🔑 PATTERN 7 — Serialize

java

```
public String serialize(TreeNode root) {
    StringBuilder sb = new StringBuilder();
    serializeHelper(root, sb);
    return sb.toString();
}

private void serializeHelper(TreeNode node, StringBuilder sb) {
    if (node == null) {
        sb.append("#").append(","); // null marker
        return;
    }
    sb.append(node.val).append(","); // write value
    serializeHelper(node.left, sb); // write left subtree
    serializeHelper(node.right, sb); // write right subtree
}
```

🔑 PATTERN 7 — Deserialize

java

```
public TreeNode deserialize(String data) {  
    Queue<String> queue = new LinkedList<>(Arrays.asList(data.split(",")));  
    return deserializeHelper(queue);  
}  
  
private TreeNode deserializeHelper(Queue<String> queue) {  
    String val = queue.poll(); // read next value  
  
    if (val.equals("#")) return null; // null marker  
  
    TreeNode node = new TreeNode(Integer.parseInt(val));  
    node.left = deserializeHelper(queue); // rebuild left subtree  
    node.right = deserializeHelper(queue); // rebuild right subtree  
    return node;  
}
```

How to Identify the Pattern

Read the problem. Ask these questions in order:

START

|



Is it a BST (left < node < right)?

|— YES → PATTERN 4 (BST Properties)

| Use in-order for sorted order

| Use min/max bounds for validation

|— NO ↓

Does the question mention "levels" or "layer by layer"?

|— YES → PATTERN 3 (BFS)

| Use Queue + levelSize trick

|— NO ↓

Does the question involve TWO trees being compared?

|— YES → PATTERN 2 (Comparing Two Trees)

| Check null cases first, then values, then recurse

|— NO ↓

Does the question ask for a PATH sum (not necessarily through root)?

|— YES → PATTERN 5 (Path Problems + Global Max)

| Use global variable, return single-direction gain

|— NO ↓

Does the question ask to BUILD a tree from traversal arrays?

|— YES → PATTERN 6 (Tree Construction)

| First element of preorder = root, use inorder to split

|— NO ↓

Does the question ask to SAVE and RESTORE a tree?

|— YES → PATTERN 7 (Serialization)

| Pre-order DFS + null markers

|— NO ↓

Default → PATTERN 1 (Simple DFS, bottom-up return value)

Ask: "What does a node need from its children to compute its answer?"

Complexity Cheat Sheet

Pattern	Time	Space	Notes
Pattern 1 — Simple DFS	$O(n)$	$O(h)$	h = height; $O(\log n)$ balanced, $O(n)$ skewed
Pattern 2 — Compare Trees	$O(n \cdot m)$ worst	$O(h)$	Subtree check; $O(n)$ for same tree
Pattern 3 — BFS	$O(n)$	$O(n)$	Queue holds max one full level
Pattern 4 — BST	$O(h)$ to $O(n)$	$O(h)$	Faster on balanced BST
Pattern 5 — Path Sum	$O(n)$	$O(h)$	Global variable updated at each node
Pattern 6 — Construction	$O(n)$	$O(n)$	HashMap for $O(1)$ inorder lookups
Pattern 7 — Serialize	$O(n)$	$O(n)$	String stores all nodes + nulls

h = height of tree. For a balanced tree $h = O(\log n)$. For a skewed tree (linked list shape) $h = O(n)$.

Common Mistakes to Avoid

✗ Mistake 1: Forgetting the null base case

```
java
// WRONG - will throw NullPointerException
int maxDepth(TreeNode root) {
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}

// CORRECT - always check null first
int maxDepth(TreeNode root) {
    if (root == null) return 0; // ← THIS LINE SAVES YOU
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

✗ Mistake 2: BST validation — only checking immediate children

```
java
```

```
// WRONG – misses nodes that violate global BST property
boolean isValidBST(TreeNode root) {
    if (root == null) return true;
    if (root.left != null && root.left.val >= root.val) return false;
    if (root.right != null && root.right.val <= root.val) return false;
    return isValidBST(root.left) && isValidBST(root.right);
}

// CORRECT – pass min/max bounds
boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
```

✗ Mistake 3: Ignoring negative values in path sum

```
java

// WRONG – adds negative gains which reduce sum
int leftGain = maxGain(node.left);
int rightGain = maxGain(node.right);

// CORRECT – use 0 if child contribution is negative
int leftGain = Math.max(maxGain(node.left), 0);
int rightGain = Math.max(maxGain(node.right), 0);
```

✗ Mistake 4: Returning wrong thing from diameter helper

```
java

// WRONG – returning diameter (leftH + rightH) to parent
// Parent needs HEIGHT not diameter!
return leftH + rightH;

// CORRECT – return HEIGHT, store diameter in global
globalMax = Math.max(globalMax, leftH + rightH); // store diameter
return 1 + Math.max(leftH, rightH);              // return height
```

✗ Mistake 5: Using int instead of long for BST validation

```
java
```

```
// WRONG – Integer.MIN_VALUE can cause edge case failures
boolean validate(TreeNode node, int min, int max) { ... }

// CORRECT – use long to handle all int values safely
boolean validate(TreeNode node, long min, long max) { ... }
// Call with: validate(root, Long.MIN_VALUE, Long.MAX_VALUE)
```

Quick Reference Card

TREE PATTERN QUICK CARD	
PATTERN 1 (Most common)	DFS bottom-up return value if null → base recurse left & right → combine → return
PATTERN 2	Compare two trees both null → true one null → false values differ → false recurse both
PATTERN 3	BFS with Queue offer root while queue: size=q.size loop size times: poll, process, offer children
PATTERN 4	BST: use left < node < right property In-order → sorted pass min/max bounds
PATTERN 5	Path sum: global variable gain = Math.max(childGain, 0) update global return node.val + max(L,R)
PATTERN 6	Build from preorder + inorder preorder[0]=root find root in inorder split → recurse left & right
PATTERN 7	Serialize: preorder DFS + "#" for null Deserialize: Queue, read one by one

Problem → Pattern Mapping

Problem	Pattern	Key Idea
Invert Binary Tree	Pattern 1	Swap children, recurse
Maximum Depth of Binary Tree	Pattern 1	$1 + \max(\text{left}, \text{right})$
Diameter of Binary Tree	Pattern 1 + Global	Height helper + global max
Balanced Binary Tree	Pattern 1 + Sentinel	Return -1 for unbalanced
Same Tree	Pattern 2	Compare node by node
Subtree of Another Tree	Pattern 2	isSameTree at every node
Binary Tree Level Order Traversal	Pattern 3	BFS with level grouping
Binary Tree Right Side View	Pattern 3	Last node of each level
Lowest Common Ancestor of BST	Pattern 4	Split left/right/at root
Validate Binary Search Tree	Pattern 4	Pass min/max bounds
Kth Smallest Element in a BST	Pattern 4	In-order traversal count
Count Good Nodes In Binary Tree	Pattern 1	Track max on path from root
Binary Tree Maximum Path Sum	Pattern 5	Global max + gain from children
Construct Binary Tree from Traversal	Pattern 6	Preorder root + inorder split
Serialize and Deserialize Binary Tree	Pattern 7	Preorder + null markers

Notes Version 1.0 — Cover all 15 NeetCode Tree problems with 7 patterns. Revise: Pattern 1 → 3 times, Pattern 3 BFS → 2 times, Pattern 4 BST → 2 times